CSE 4502: Operating System Lab
Lab 1: The Command Line Essentials

Jibon Naher, Lecturer, CSE

February 26, 2026

Introduction

This assignment is designed to provide essential hands-on experience with **20 fundamental commands** used for creating, viewing, manipulating, and managing files and directories in a Ubuntu (Linux) environment. All commands must be executed within your system's terminal.

---

1 Part 1: Preparation & Setup (4 Commands)

Objective

Create a dedicated workspace and practice navigating the file system.

Tasks

- Create a primary directory named `linux_assignment`.
- Navigate into this new directory.
- Create a sub-directory named `Documents`.

Table 1: Commands for Setup

No.	Command	Description
1	`pwd`	Print Working Directory (Verify your current location).
2	`mkdir`	Make Directory (Create `linux_assignment` and `Documents`).
3	`cd`	Change Directory (Move into `linux_assignment` and navigate between folders).
4	`ls`	List directory contents (Use `ls -l` to see file permissions and details).

2 Part 2: File Creation and Viewing (5 Commands)

Objective

Create several files and practice viewing their contents and metadata efficiently.

Tasks

- Create two empty files: `report.txt` and `notes.log`.
- Add a line of text (e.g., "Experiment data collected on 2024-01-01") to a third file, `data.csv`, using a single command line redirection.
- Create a large, empty file (e.g., 10MB) to practice paging.

Table 2: Commands for Creation and Viewing

No.	Command	Description
5	`touch`	Create a new, empty file or update the timestamp of an existing one.
6	`echo`	Output text (Used with redirection > to populate `data.csv`).
7	`cat`	Concatenate and display the entire file content (View `data.csv`).
8	`file`	Determine the file type (Check `report.txt`).
9	`less`	Pager utility to view files one page at a time (Use for the large file).

3 Part 3: Copying, Moving, and Renaming (4 Commands)

Objective

Practice moving and copying files/directories and renaming objects.

Tasks

- Copy `report.txt` to the `Documents` sub-directory.
- Rename `notes.log` to `server_logs.txt` in the current directory.
- Move `data.csv` into the `Documents` sub-directory.
- Create a temporary empty directory called `temp` and immediately remove it.

Table 3: Commands for Manipulation

No.	Command	Description
10	`cp`	Copy files.
11	`mv`	Move or rename files and directories.
12	`cp -r`	Copy directories recursively (Create a new directory `backup` and copy `Documents` into it).
13	`rmdir`	Remove an empty directory.

4 Part 4: Searching and Permissions (5 Commands)

Objective

Locate files, search for content, manage file sizes, and handle access permissions.

Tasks

- Search for the location of `server_logs.txt` starting from your home directory.
- Search for the text "Experiment" *within* `Documents/data.csv`.
- Check the total disk space used by the `linux_assignment` directory in human-readable format.
- Change the permissions of `server_logs.txt` using the numeric mode `600` (read/write only for the user).

Table 4: Commands for Searching, Size, and Permissions

No.	Command	Description
14	`find`	Search for files in a directory hierarchy based on criteria.
15	`grep`	Search for a specific text pattern within file content.
16	`chmod`	Change file access permissions.
17	`chown`	Change file owner and/or group.
18	`du`	Disk Usage (Check the size of a directory in human-readable format, `du -h`).

5 Part 5: Advanced Viewing and Cleanup (2 Commands)

Objective

View partial file content and remove the entire assignment structure.

Tasks

- View only the first 5 lines of `server_logs.txt`.
- Remove the entire `linux_assignment` directory, including all sub-directories (`Documents`, `backup`) and files.

Table 5: Commands for Final Steps

No.	Command	Description
19	`head`	Output the beginning (first part) of files.
20	`rm`	Remove files or directories (Use the recursive option, `rm -r`, for directory removal).

Submission Requirements

1. **List of Commands:** A numbered list of all 20 commands used (matching the numbers in the tables).

2. **Syntax:** The exact command line syntax you used for each command (e.g., `touch report.txt`).
3. **Execution Proof:** A single, clear, scrolling screenshot (or multiple combined screenshots) showing the execution and output of **all 20 commands** performed sequentially in your terminal. This demonstrates successful completion of the entire assignment flow.

CSE 4502: Operating System Lab
Lab 2: Intermediate Linux Commands

Jibon Naher, Lecturer, CSE

February 26, 2026

Introduction

This lab follows Lab 1 and introduces **30 intermediate and advanced commands** essential for text processing, data compression, system monitoring, and process management. Successful completion requires demonstrating the use of command chaining and I/O control.

Introductory Survey - 20 minutes

1. Please go to the link: Introductory Student Survey 2. Fill up and submit the survey

1 Part 1: Preparation, Help, and System Information (6 Commands)

Objective

Set up the workspace, utilize built-in help features, and retrieve basic system information.

Tasks

- Create a main directory `lab2` and a sub-directory `text_data`.
- Use the appropriate command to display the manual page for the `tar` command.
- Display the last 10 commands you executed.

2 Part 2: Text Processing and Filters (8 Commands)

Objective

Practice filtering, counting, sorting, and transforming text data.

Table 1: Commands for Help and Metadata

No.	Command	Description
1	`mkdir`	Create the workspace (`lab2` and `text_data`).
2	`cd`	Navigate into the `lab2` directory.
3	`man`	Display the manual page for a command (e.g., `man tar`).
4	`history`	View the list of previously executed commands.
5	`stat`	Display detailed file status and metadata (Check metadata for `text_data`).
6	`uname`	Print system information (e.g., kernel name/version).

Tasks

- Create two simple text files (`fileA.txt`, `fileB.txt`) with minor differences.
- Combine the content of both files, sort the lines, and count the total lines.
- Remove duplicate lines from a file after sorting it.

Table 2: Commands for Text Manipulation

No.	Command	Description
7	`wc`	Word Count (count lines, words, and bytes).
8	`sort`	Sort lines of text files.
9	`uniq`	Report or omit repeated lines (use after `sort`).
10	`diff`	Find the differences between two files.
11	`cut`	Remove sections from each line of files (e.g., specific column data).
12	`tr`	Translate or delete characters (e.g., change all lowercase to uppercase).
13	`sed`	Stream Editor (Perform a basic substitution, e.g., change 'old' to 'new' in a file).
14	`more`	Pager utility to view file content page by page (Use for a large output).

3 Part 3: Archiving, Compression, and Linking (6 Commands)

Objective

Compress data for storage and sharing, and understand file linking.

Tasks

- Create an archive of the entire `text_data` folder and compress it using `.tar.gz` format.
- Decompress and extract the archive into a new directory `extracted_data`.
- Create a symbolic link (`-s`) to a file, and a hard link to another file.

Table 3: Commands for Archiving and Links

No.	Command	Description
15	`tar -czf`	Create a compressed archive (gzip format).
16	`tar -xf`	Extract files from an archive.
17	`gzip`	Compress a single file using the GZIP algorithm.
18	`gunzip`	Decompress a file previously compressed with `gzip`.
19	`ln -s`	Create a symbolic (soft) link to a file.
20	`ln`	Create a hard link to a file.

4 Part 4: System Status and Process Control (6 Commands)

Objective

Monitor system resource utilization and manage running processes.

Tasks

- Display the available and used disk space on your mounted file systems.
- View the memory usage statistics (RAM and Swap).
- Run the live process monitor and observe system performance for a few seconds.
- Use a command to display the network configuration of your interfaces.
- Use a command to find the process ID (PID) of a command you run in the background, and then terminate it.

Table 4: Commands for System Monitoring

No.	Command	Description
21	`df`	Disk Free (report file system disk space usage).
22	`free`	Display amount of free and used memory (RAM and swap).
23	`top`	Display Linux processes (Interactive system-monitor).
24	`ps`	Report a snapshot of the current processes.
25	`kill`	Send a signal (terminate) to a process using its PID.
26	`ip addr`	Display network interface addresses (Modern replacement for `ifconfig`).

5 Part 5: I/O Control, Aliases, and Utilities (4 Commands)

Objective

Master command chaining, output redirection, and customize your environment.

Tasks

- Use the pipe (|) to chain the output of one command to the input of another (e.g., `history | grep`).
- Use output redirection (>>) to append a message to an existing file.
- Create a temporary alias for a long command, test it, and then remove it.
- Display the system time and date, and then the calendar for the current month.

Table 5: Commands for I/O and Utilities

No.	Command	Description
27	`alias / unalias`	Create and remove command shortcuts.
28	`>>`	I/O Redirection operator (Append output to a file).
29	`|`	Pipe operator (Chain commands by passing output as input).
30	`date / cal`	Print the system date/time and display a calendar.

Submission Requirements

1. **List of Commands:** A numbered list of all 30 commands used (matching the numbers in the tables).
2. **Syntax:** The exact command line syntax you used for each command (e.g., `tar -czf archive.tar.gz text_data/`).
3. **Execution Proof:** A single, clear, scrolling screenshot (or multiple combined screenshots) showing the execution and output of **at least 10 commands**(randomly chosen) performed sequentially in your terminal. This shows the successful completion of the entire assignment flow.

Lab3 Task 1A

Jibon Naher[1] and Samnun Azfar[2]

[1]Lecturer,CSE
[2]Junior Lecturer,CSE

February 26, 2026

1 Check Even or Odd

This script demonstrates reading user input, performing an arithmetic operation (modulo), and using conditional logic to determine if a number is even or odd.

1.1 Algorithm

1. Start the program.
2. Read the value of n (the number).
3. Calculate the remainder r: $r = n\%2$.
4. If the value of r equals 0 then print the number is even.
5. If the value of r not equal to 0 then print the number is odd.

2 Check Prime Number

This script demonstrates using loops, arithmetic operations, and conditional logic to determine whether a given number is prime. A prime number is a positive integer greater than 1 that has no positive divisors other than 1 and itself.

2.1 Algorithm

1. Start the program.
2. Read the value of n (the number).
3. If $n \leq 1$, then it is not a prime number.
4. Initialize a counter variable `i=2`.
5. Repeat while $i \leq n/2$:
   (a) Calculate: r: $r = n\%2$.
   (b) If $r = 0$, then n is not a prime number; terminate the loop.
   (c) Increment i by 1.
6. If no divisor was found, the number is prime.

Lab3 Task 1B

Jibon Naher[1] and Samnun Azfar[2]

[1]Lecturer,CSE
[2]Junior Lecturer,CSE

February 26, 2026

1 Check Whether a Number is Positive, Negative, or Zero

This script demonstrates the use of conditional statements to classify a number based on its sign.

1.1 Algorithm

1. Start the program.
2. Read the value of `n`.
3. If `n > 0`, display "The number is positive."
4. Else if `n < 0`, display "The number is negative."
5. Else, display "The number is zero."

2 Calculate the Sum of First N Natural Numbers

This script demonstrates the use of loops and arithmetic operations to calculate the sum of the first N natural numbers.

2.1 Algorithm

1. Start the program.
2. Read the value of n.
3. Initialize `sum=0` and `i=1`.
4. Repeat while `i <= n`:
   (a) `sum = sum + i`.
   (b) Increment `i = i + 1`.
5. Display the final value of `sum`.

Lab3 Task 2A

Jibon Naher[1] and Samnun Azfar[2]

[1]Lecturer,CSE
[2]Junior Lecturer,CSE

February 26, 2026

1 Find the Largest of Three Numbers

This script demonstrates the use of conditional statements (`if`, `elif`, `else`) to find the largest among three given numbers.

1.1 Algorithm

1. Start the program.
2. Read three numbers: `a, b, c`.
3. Compare:
   (a) If `a > b` and `a > c`, then `a` is the largest.
   (b) Else if `b > c`, then `b` is the largest.
   (c) Else `c` is the largest.
4. Display the largest number.

2 Display Fibonacci Series

This script demonstrates using loops, variables, and arithmetic operations to generate a sequence of Fibonacci numbers. The Fibonacci sequence starts with 0 and 1, and each subsequent number is the sum of the previous two.

2.1 Algorithm

1. Start the program.
2. Read the number of terms, n.
3. Initialize `a=0, b=1`.
4. Print `a` and `b`.
5. Repeat $(n-2)$ times:
   (a) `c = a + b`.
   (b) Print `c`.
   (c) Update: `a = b`, `b = c`.

Lab3 Task 2B

Jibon Naher[1] and Samnun Azfar[2]

[1]Lecturer,CSE
[2]Junior Lecturer,CSE

February 26, 2026

1 Task 1: Check Whether a Year is Leap Year or Not

This script demonstrates conditional logic and modulus arithmetic to check leap year rules.

1.1 Algorithm

1. Start the program.
2. Read the year value `y`.
3. If `y % 400 == 0`, then it is a leap year.
4. Else if `y % 100 == 0`, then it is not a leap year.
5. Else if `y % 4 == 0`, then it is a leap year.
6. Else, it is not a leap year.
7. Display the result.

2 Task 2: Find the Factorial of a Number

This script demonstrates using loops and arithmetic operations to compute the factorial of a given number. The factorial of a number n (written as n!) is the product of all positive integers less than or equal to n.

2.1 Algorithm

1. Start the program.
2. Read the value of n.
3. Initialize `fact=1`.
4. Repeat while $n > 0$:
   (a) Multiply `fact = fact * n`.
   (b) Decrement `n = n - 1`.
5. Print the value of `fact`.

Lab3 Task 1A

Jibon Naher[1] and Samnun Azfar[2]

[1]Lecturer,CSE
[2]Junior Lecturer,CSE

February 26, 2026

1 Check Even or Odd

This script demonstrates reading user input, performing an arithmetic operation (modulo), and using conditional logic to determine if a number is even or odd.

1.1 Algorithm

1. Start the program.
2. Read the value of n (the number).
3. Calculate the remainder r: $r = n\%2$.
4. If the value of r equals 0 then print the number is even.
5. If the value of r not equal to 0 then print the number is odd.

2 Check Prime Number

This script demonstrates using loops, arithmetic operations, and conditional logic to determine whether a given number is prime. A prime number is a positive integer greater than 1 that has no positive divisors other than 1 and itself.

2.1 Algorithm

1. Start the program.
2. Read the value of n (the number).
3. If $n \leq 1$, then it is not a prime number.
4. Initialize a counter variable `i=2`.
5. Repeat while $i \leq n/2$:
   (a) Calculate: r: $r = n\%2$.
   (b) If $r = 0$, then n is not a prime number; terminate the loop.
   (c) Increment i by 1.
6. If no divisor was found, the number is prime.

3 Find the Factorial of a Number

This script demonstrates using loops and arithmetic operations to compute the factorial of a given number. The factorial of a number n (written as n!) is the product of all positive integers less than or equal to n.

3.1 Algorithm

1. Start the program.
2. Read the value of n.
3. Initialize `fact=1`.
4. Repeat while $n > 0$:
   (a) Multiply `fact = fact * n`.
   (b) Decrement `n = n - 1`.
5. Print the value of `fact`.

4 Display Fibonacci Series

This script demonstrates using loops, variables, and arithmetic operations to generate a sequence of Fibonacci numbers. The Fibonacci sequence starts with 0 and 1, and each subsequent number is the sum of the previous two.

4.1 Algorithm

1. Start the program.
2. Read the number of terms, n.
3. Initialize `a=0, b=1`.
4. Print `a` and `b`.
5. Repeat $(n-2)$ times:
   (a) `c = a + b`.
   (b) Print `c`.
   (c) Update: `a = b`, `b = c`.

5 Find the Largest of Three Numbers

This script demonstrates the use of conditional statements (`if`, `elif`, `else`) to find the largest among three given numbers.

5.1 Algorithm

1. Start the program.
2. Read three numbers: `a, b, c`.
3. Compare:
   (a) If `a > b` and `a > c`, then `a` is the largest.
   (b) Else if `b > c`, then `b` is the largest.
   (c) Else `c` is the largest.
4. Display the largest number.

6 Check Whether a Year is Leap Year or Not

This script demonstrates conditional logic and modulus arithmetic to check leap year rules.

6.1 Algorithm

1. Start the program.
2. Read the year value `y`.
3. If `y % 400 == 0`, then it is a leap year.
4. Else if `y % 100 == 0`, then it is not a leap year.
5. Else if `y % 4 == 0`, then it is a leap year.
6. Else, it is not a leap year.
7. Display the result.

Basic Shell Scripting Reference Guide

Jibon Naher[1] and Samnun Azfar[2]

[1]Lecturer, CSE
[2]Junior Lecturer,CSE

February 26, 2026

1 Shell?

1.1 What Is a Shell?

A *shell* is a user interface that provides a way to interact with the operating system. It acts as a bridge between the user and the system resources managed by the kernel. The shell itself is an executable program (commonly written in languages such as C) that can interpret user commands, execute programs, and run scripts.

A shell typically performs the following tasks:

- Reads user input (commands).
- Parses and interprets the commands.
- Invokes the appropriate executables or built-in functions.
- Delegates program execution requests to the kernel.
- Displays output or error messages back to the user.

$$\textit{User Input/Script} \rightarrow \textit{Shell} \rightarrow \textit{Kernel} \rightarrow \textit{System Calls / Hardware Resources.}$$

There are two major categories of shells:

- **Command-Line Shells:** Provide a text-based interface. Examples include:
  - `sh` — The original Bourne shell.
  - `bash` — Bourne Again SHell; popular on Linux.
  - `zsh` — Extended Bourne shell with more features.
  - `fish` — User-friendly, modern interactive shell.
  - `csh` / `tcsh` — C-style syntax shells.
- **Graphical Shells:** Provide a GUI for interacting with the system. Examples include:
  - GNOME Shell
  - KDE Plasma Shell
  - Windows Explorer (conceptually a GUI shell)

Regardless of type, all shells serve the same fundamental purpose: enabling the user to control and interact with the system through the operating system.

1.2 How Do We Reach a Shell Prompt?

At a hardware level, a computer system consists of the motherboard, CPU, RAM, and I/O devices. The operating system orchestrates the interactions between these components. In this lab, we aim to understand how the OS, after initializing the hardware and its own internal subsystems, eventually exposes a *shell* to the user. Earlier labs explored executing commands; this section formalizes what a shell is and how the system reaches it.

Let us assume we are booting into a Linux system. The process typically goes as follows:

- The *firmware* (stored on the motherboard—traditionally BIOS, now often UEFI) searches for a *bootloader*. The bootloader usually resides on a disk's EFI System Partition (for UEFI) or in the Master Boot Record (for legacy BIOS). The firmware knows where to look based on its configuration.
- Once the bootloader is loaded into RAM, it begins execution. Bootloaders (such as GRUB) are small programs, often with some components written in assembly. They are commonly structured in multiple stages, but we do not go into those details here.
- The bootloader's primary responsibility is to load the operating system kernel (an executable binary, typically written in C with some assembly). It places the kernel into memory and transfers control to it.
- The kernel initializes core subsystems: memory management, process scheduler, interrupt handlers, and essential device drivers (including storage, input, and display subsystems).
- After initialization, the kernel executes the first userspace program, typically `init` or `systemd`.
- The system and service manager (`systemd`), running as PID 1, continues the boot process by starting required system services.
- One of these services is the login process (such as `agetty` on a TTY or a graphical login manager in GUI environments).
- After a successful login, `systemd` or the login process launches the user's default *shell* (e.g., `bash`, `zsh`, or `fish`). The shell then provides the user with an interactive interface to execute commands and control the system.

Note that the steps in subsection 1.2 occur regardless of whether the system ultimately presents a GUI. Modern Linux distributions typically start a graphical interface in addition to the shell, but the underlying boot sequence remains the same.

1.3 BASH (Bourne Again SHell)

BASH stands for **Bourne Again SHell**, which is the default shell for most Linux distributions and macOS. It is an enhanced replacement for the original Bourne Shell (sh).

1.3.1 Components of BASH

BASH consists of several key components that work together to provide a complete command-line environment:

1. **Commands**: Built-in commands like `cd`, `echo`, `pwd`, `export`, etc.

2. **Executables**: External programs stored in directories like `/bin`, `/usr/bin`, `/usr/local/bin`
3. **Keywords**: Reserved words like `if`, `then`, `else`, `fi`, `for`, `while`, `do`, `done`
4. **Scripts**: Text files containing sequences of commands with `.sh` extension
5. **Variables**: User-defined and environment variables (`$PATH`, `$HOME`, `$USER`)
6. **Functions**: Reusable code blocks that can be defined within scripts or interactively

The command `type` lets us check the type, if it's a keyword,command or an executable.

```
type ls # ls is an executable, returns the description
```

```
type cd # cd is a command as it changes the state of the shell
```

```
type if # if is a keyword used in branching statements
```

2 Creating and Running a Shell Script

Step 1 Create a script file

Use a text editor (like `gedit` or `nano`):

```
gedit hello.sh
```

Step 2 Add the script content

```
#!/bin/bash
# This is a comment
echo "Hello, World!"
```

Observe the `#!/bin/bash` line at the start of the script. The first two ascii characters- `#!` tells the shell that the following parts of the first line is the path to the BASH interpreter. The script file is then passed to the BASH interpreter then the rest of the execution is done through the interpreter itself while the shell we are working on waits.

Step 3 Make it executable

```
chmod +x hello.sh # +x gives the script permission to execute
```

Step 4 Run the script

```
./hello.sh
```

Output: `Hello, World!`

3 Variables in Shell

You can store and use values easily.

variables.sh

```
#!/bin/bash
name="Max"
echo "Hello, $name!"
```

- Always use double quotation when performing variable expansion.
- No spaces around = when assigning values.

4 Environment Variables

Environment variables exist at the process level. They are not a feature of the bash shell, but rather a feature of any program

5 Conditional Statements

conditional.sh

```
#!/bin/bash
num=10
if [ $num -gt 5 ]
then
  echo "Number is greater than 5"
else
  echo "Number is less than or equal to 5"
fi
```

Operator	Meaning
`-eq`	equal
`-ne`	not equal
`-gt`	greater than
`-lt`	less than
`-ge`	greater or equal
`-le`	less or equal

Table 1: Numeric comparison operators for `[ ... ]` and `[[ ... ]]`

- The `[[ ... ]]` is the replacement of the depriciated `test` command. `[[` acts as a command whose last parameter is `]]` and within the brackets you are able to execute numeric comparisons which are returned as a boolean result utilized in the `if` statement.
- The `` ` = ` `` operator(with spaces around) is used to compare strings, it cannot compare number.
- The operators mentioned in Table 1 are numeric operators only working on numbers.

6 Arrays

Bash supports one-dimensional indexed arrays, which can store multiple values under a single variable name. Arrays can be created explicitly or implicitly by assignment, and elements are accessed using their index.

Declaring Arrays

You can declare arrays with `declare -a` or by simple assignment.

```
# Declaring with initial values
fruits=("apple" "banana" "mango")

# Implicit declaration
colors[0]="red"
colors[1]="blue"
```

Accessing and Expanding Arrays

Array elements are accessed with `$array[index]`. To expand all elements, use `$array[@]` or `$array[*]`.

```
echo "${fruits[0]}"        # prints: apple

echo "${fruits[@]}"        # prints all elements
echo "${#fruits[@]}"       # number of elements
```

Appending to Arrays

Use += or assign to the next index.

```
fruits+=("orange")         # append element

fruits[${#fruits[@]}]="grape"   # append manually

echo "${fruits[@]}"
```

Generating Arrays

You can generate arrays using command substitution, brace expansion, or loops.

```
# From command output
files=( $(ls *.txt) )

# Numeric sequence using brace expansion
nums=({1..10})

# Generated in a loop
for i in {1..5}; do
    squares+=($((i*i)))
done

echo "${squares[@]}"   # 1 4 9 16 25
```

7 Loops

7.1 For Loop

```
for i in 1 2 3 4 5;
do
  echo "Number: $i"
done
```

7.2 While Loop

```
count=1
while [ $count -le 5 ];
do
  echo "Count = $count"
  count=$((count + 1))
done
```

8 Reading User Input

read_input.sh

```
#!/bin/bash
echo "Enter your name:"
read name
echo "Welcome, $name!"
```

9 Command-Line Arguments

You can pass values to scripts from the terminal.

arguments.sh

```
#!/bin/bash
echo "First argument: $1"
echo "Second argument: $2"
echo "Total arguments: $#"
```

Run: `./myscript.sh apple banana`
Output:

- First argument: apple
- Second argument: banana
- Total arguments: 2

10 Using System Commands

Shell scripts can automate system tasks easily.

system_commands.sh

```
#!/bin/bash
echo "Today's date is:"
date
echo "Current directory is:"
pwd
echo "Files in this directory:"
ls -lh
```

11 Exit Status and Comments

- `#` → used for comments
- `$?` → shows exit status of the last command (`0` = success)

11.1 Example

```
ls /non/existent/path
echo $?
```

(This will likely output a number other than 0, indicating an error.)

12 Arithmetic Expressions

Bash provides built-in support for arithmetic evaluation, which allows you to perform integer calculations directly inside the shell. The most common forms are `$(( ... ))` for substitution and `(( ... ))` for arithmetic evaluation.

- `$((expression))` returns the result and can be assigned or printed.
- `((expression))` evaluates the expression and returns an exit status (0 if true, 1 if false), making it useful in conditions.

Examples:

```
# Arithmetic substitution
x=$((3 + 5))
echo $x   # prints 8

# Increment
((x++))

# Using arithmetic evaluation in conditions
if (( x > 5 )); then
    echo "x is greater than 5"
fi
```

Lab-5 1A

Jibon Naher[1] and Samnun Azfar[2]

[1]Lecturer, CSE
[2]Junior Lecturer,CSE

February 26, 2026

Part A (Mutex): Multi-Account Transfer System

Problem Statement
You are given three shared bank accounts with initial balances: $A = 1000, B = 1000, C = 1000$. Three threads run concurrently to perform transfers.

Input (Workload)

- **Thread T1:** Transfers 100 from $A \to B$ (10 times).
- **Thread T2:** Transfers 50 from $B \to C$ (20 times).
- **Thread T3:** Transfers 25 from $C \to A$ (40 times).

Constraints

- A transaction is one instance of money being transferred between accounts, that implies **Thread T1** executes 10 transactions.
- You can use **multiple mutexes**. One for each account.
- Transfers must be **all-or-nothing**.
- No account may ever go negative.

Expected Output

- **Intermediate:** Logs showing transfer start/end without interleaved errors. After each thread is done executing print the balances of the accounts.
- **Final State:** $A = 1000, B = 1000, C = 1000$. (Note: All transfers technically cancel out in sum).
- **Invariant:** $A + B + C = 3000$ at every step.

Part B (Semaphore): Limited-Capacity Transaction Server

Problem Statement
A transaction server can process at most 3 transactions concurrently.

Input

- 6 client threads ($C_1 \ldots C_6$) arriving almost simultaneously.
- Each client requests access, holds it for 1 second, then leaves.

Constraints

- Use **one counting semaphore** initialized to 3.

Expected Output A line showing 6 boolean values where `C1=True` indicates C1 is being served. Use a mutex, a thread to monitor and a boolean array to track the status of the threads being served. Use delay funcitons to make the console printing readable.

```
C1=True, C2=True, C3=True, C4=False, C5=False, C6=False
C1=False, C2=True, C3=True, C4=True, C5=False, C6=False
```

Delay Functions

To simulate lengthy service times in concurrent threads, explicit delay functions are introduced so that a thread occupies the server for a realistic duration before releasing it.

On Linux systems, sleep-based system calls are commonly used. The required headers and examples are shown below.

```
#include <unistd.h>     // sleep(), usleep()
#include <time.h>       // nanosleep()
```

Basic delays can be implemented as:

```
sleep(2);               // delay for 2 seconds
usleep(500000);         // delay for 500 milliseconds
```

For higher precision, `nanosleep()` provides finer control:

```
struct timespec ts;
ts.tv_sec = 1;
ts.tv_nsec = 200000000; // 200 ms
nanosleep(&ts, NULL);
```

These delay mechanisms enable realistic modeling of service duration, contention, and waiting behavior in multi-threaded simulations.

Lab-5 2A

Jibon Naher[1] and Samnun Azfar[2]

[1]Lecturer, CSE
[2]Junior Lecturer,CSE

February 26, 2026

Part A (Mutex): Priority Logging System

Problem Statement
Concurrent threads print High or low priority log messages. High-priority messages must always be processed/printed before Low-priority ones, regardless of arrival time within the critical window.

Input (Thread Actions)

- Thread 1 (Low): Prints "Network OK"
- Thread 2 (High): Prints "Disk Failure"
- Thread 3 (Low): Prints "User Login"
- Thread 4 (High): Prints "Power Surge"

Constraints

- Use **one mutex**.

Expected Output (Buffer Dump)

```
[HIGH] Disk Failure
[HIGH] Power Surge
[LOW] Network OK
[LOW] User Login
```

Part B (Semaphore): Ordered Startup System

Problem Statement
Three subsystems (Power, Communication, Control) must initialize in strict order.

Input

- Threads for Power, Comm, and Control start in random order (e.g., Control starts first, then Comm, then Power).

Constraints

- Use semaphores as signals. No mutexes.

Expected Output

```
[Control Thread] Waiting for Signal...
[Comm Thread] Waiting for Signal...
[Power Thread] Power Initialized. Signal sent.
[Comm Thread] Communication Initialized. Signal sent.
[Control Thread] Control System Initialized.
```

Delay Functions

To simulate lengthy service times in concurrent threads, explicit delay functions are introduced so that a thread occupies the server for a realistic duration before releasing it.

On Linux systems, sleep-based system calls are commonly used. The required headers and examples are shown below.

```
#include <unistd.h>     // sleep(), usleep()
#include <time.h>       // nanosleep()
```

Basic delays can be implemented as:

```
sleep(2);               // delay for 2 seconds
usleep(500000);         // delay for 500 milliseconds
```

For higher precision, `nanosleep()` provides finer control:

```
struct timespec ts;
ts.tv_sec = 1;
ts.tv_nsec = 200000000; // 200 ms
nanosleep(&ts, NULL);
```

These delay mechanisms enable realistic modeling of service duration, contention, and waiting behavior in multi-threaded simulations.

Interprocess Communication

Jibon Naher[1] and Samnun Azfar[2]

[1]Lecturer, CSE
[2]Junior Lecturer,CSE

February 26, 2026

1 Race Condition

1.1 What Is a Race Condition?

A race condition occurs when two or more processes (or threads) access shared data concurrently, and the final result depends on the order of execution. Because the order of execution is unpredictable, the program can produce inconsistent or incorrect results.

1.2 Critical Section

The part of code accessing shared data — must be executed by only one process at a time. for example, line 8 of above code: *counter++;*

1.3 Example 1: Race Condition in C (Two Threads Incrementing a Variable)

```c
#include <stdio.h>
#include <pthread.h>

int counter = 0;  // Shared resource

void* increment(void* arg) {
    for (int i = 0; i < 100000; i++) {
        counter++;  // Critical section
    }
    return NULL;
}
int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final counter value: %d\n", counter);
    return 0;
}
```

1.4 Explanation

The following section explains the function of key lines typically found in C programs that use the `pthreads` library for thread management.

1.4.1 Line 13: `pthread_t t1, t2;`

Declares two thread identifiers (`t1` and `t2`). These are like "thread handles," used by the main program to refer to the threads later (e.g., to wait for them to finish).

1.4.2 Line 14: `pthread_create(&t1, NULL, increment, NULL);`

This is the function call to **create a new thread** that runs the `increment` function concurrently.

- `&t1`: Address of the thread ID variable where the newly created thread's identifier will be stored.
- `NULL`: Specifies the thread attributes (e.g., stack size, priority). `NULL` means default attributes are used.
- `increment`: The function pointer that the new thread will begin execution in.
- `NULL`: The single argument to be passed to the `increment` function. (In this case, none is passed.)

Note: After this line, two threads are running concurrently: the main thread (continuing in `main`) and Thread `t1` (running `increment`).

1.4.3 Line 15: `pthread_create(&t2, NULL, increment, NULL);`

Similar to the above, this creates a second thread, `t2`. Now, **three threads exist** (`main`, `t1`, and `t2`). Threads `t1` and `t2` are now competing to modify the shared resource, leading to the race condition.

1.4.4 Line 17: `pthread_join(t1, NULL);`

This is a blocking call. The main thread will pause and wait for **thread** `t1` to finish its execution (i.e., exit the `increment` function). This is crucial for ensuring the main thread does not continue until the concurrent work is complete.

1.4.5 Line 18: `pthread_join(t2, NULL);`

The main thread waits for **thread** `t2` to finish. After both `pthread_join()` calls successfully return, both worker threads are guaranteed to be finished, and it is safe for the main thread to proceed, typically by reading the final result of the shared counter.

1.5 Execute the program

command: *gcc race.c -o race -pthread* and then `./race`

2 How to fix: using mutex

```c
#include <pthread.h>
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
int counter = 0;

void* increment(void* arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}
```

3 Another Fix: Using Semaphore

```c
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>  // Required for semaphores

int counter = 0;  // Shared resource

// Declare a semaphore (initialized later)
sem_t sem;

void *increment(void *) {
    for (int i = 0; i < 100000; i++) {
        sem_wait(&sem);    // Acquire the semaphore (enter critical section)
        counter++;         // Critical section (now safe!)
        sem_post(&sem);    // Release the semaphore (exit critical section)
    }
}

int main() {
    pthread_t t1, t2;

    // Initialize semaphore to 1 (binary semaphore = mutex behavior)
    sem_init(&sem, 0, 1);

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    // Destroy the semaphore
    sem_destroy(&sem);

    printf("Final counter value: %d\n", counter);
    return 0;
}
```

Mutexes And Semaphores

Jibon Naher[1] and Samnun Azfar[2]

[1]Lecturer, CSE
[2]Junior Lecturer,CSE

February 26, 2026

1 Pointer Safety and Multiplexing

Part A (Mutex): Hot-Swappable Configuration System

Problem Statement
A server application uses a global configuration object.

- **Reader Threads:** Continuously access values from the current configuration.
- **Updater Thread:** Periodically generates a completely new configuration object and replaces the old one.

Rules

- Readers must never access invalid or freed memory.
- The switch from old config to new config must be atomic.
- Old configuration memory must be safely freed only after the swap is secured.

Constraints

- Use **one mutex** only.
- Do not use Read-Write locks.

Hints

- Use an int pointer variable as the config object.
- Write one update function which calls `free()` on the pointer variable and then uses `malloc()` to allocate new memory for the pointer. Then set the pointer to some value.
- Write multiple reader functions and run them in separate threads, then print the value of the pointer.
- Use `stdlib.h` for memory allocation functions and `unistd.h` to use the `sleep()` functions to make the printings look sane in terminal.

Invariant to Preserve

$$\text{active_config} \neq \text{NULL} \quad (\text{At all times})$$

Part B (Semaphore): Dining Philosophers (Limited Seating)

Problem Statement
5 Philosophers sit around a table with 5 forks. To eat, a philosopher needs both the left and right forks.

Deadlock Prevention Strategy
Instead of complex fork logic, you must restrict the number of philosophers allowed at the table simultaneously.

- Max **4 philosophers** allow to sit at the table at once.
- If 4 are seated, at least one is guaranteed to be able to pick up both forks.

Constraints

- Use **one counting semaphore** to represent the room capacity.
- Use mutexes/semaphores for individual forks as needed.

2 Complex Buffering and Barriers

Part A (Mutex): Thread-Safe Histogram (Sharded Updates)

Problem Statement
A large dataset is processed by multiple threads. They update a shared frequency array (histogram) of size 10.

```
histogram[10] = {0, 0, ...}
```

Rules

- Input is an array of n integers which have values k ranging from $0 < k < 10$. k is an integer.
- Assume you have $n = 9$. If you create 2 threads then thread $t1$ will calculate the histogram for indices $0 - 4$ and thread $t2$ will calculate the histogram for indices $5 - 8$, both done in a historgam array local to their threads. Then both the threads concurrently update the global histogram using the locally generated histograms.

Constraints

- Use **one mutex**.

Invariant to Preserve

$$\sum \text{histogram}[i] = n$$

Part B (Semaphore): H_2O Bond Formation (Barrier)

Problem Statement
A water molecule is formed by 2 Hydrogen atoms and 1 Oxygen atom. Threads represent individual atoms.

Rules

- H atoms must wait for another H and one O.

- O atoms must wait for two H atoms.
- They must ”bond” (exit the barrier) simultaneously in groups of three (H-H-O).
- Hint: You can concurrently create H but not O.

Constraints

- Use **semaphores only**.
- No mutexes.
- Ensure no atom is left behind or grouped incorrectly.

Expected Execution

```
H Ready -> H Ready -> O Ready -> Bond Formed!
```

Lab 6 - 1B

Jibon Naher[1] and Samnun Azfar[2]

[1]Lecturer, CSE
[2]Junior Lecturer,CSE

February 26, 2026

General Instructions

1. Use the PC from the lab.
2. Submit your mobile at the front.
3. Turn off your internet connection.
4. Your time is 50 minutes.

Part A : Sequential Launch of Space Systems

Problem Statement
In a space mission control system, three subsystems — Fuel System, Navigation System, and Launch System — must activate in a strict order for safety and synchronization.

Points to note

- Even though these systems (threads) start in random order, they must always initialize as: Fuel -> Navigation -> Launch
- Create three threads for fuel, navigation and launch.
- Use two semaphores: sem_nav -> Signals that the fuel system is ready, sem_launch -> Signals that navigation is ready.

Expected Output

```
[Navigation Thread] Waiting for fuel system...
[Launch Thread] Waiting for navigation system...
[Fuel Thread] Fuel system activated. Signal sent.
[Navigation Thread] Navigation system online. Signal sent.
[Launch Thread] Launch sequence initiated!
```

PART B: Readers–Writers Synchronization Problem

In a database system, multiple processes may either read or write shared data. To maintain data consistency, the following conditions must hold: Multiple readers can read the shared data simultaneously, Only one writer can modify the data at a time, When a writer is writing, no reader may access the data.

Your task is to simulate this behavior using threads and semaphores to ensure proper synchronization between readers and writers.

Input Description

- Create multiple reader and writer threads that access a shared variable (e.g., data). Readers should display the current value of data. Writers should increment or update the shared variable.
- All threads start in random order, but synchronization must ensure that: Multiple readers can read concurrently and Only one writer can write at a time.

Hints

- Use an integer variable read_count initialized to 0.
- Use two semaphores: mutex -> to protect the read_count variable, write_block -> to block writers when readers are active, and vice versa.

Expected Output

```
[Reader 1] Waiting to read...
[Reader 2] Waiting to read...
[Writer 1] Waiting to write...
[Reader 1] Reading data: 0
[Reader 2] Reading data: 0
[Writer 1] Writing data...
[Reader 3] Waiting to read...
[Writer 1] Updated data to 1
[Reader 3] Reading data: 1
```

Lab 6 - 2B

Jibon Naher[1] and Samnun Azfar[2]

[1]Lecturer, CSE
[2]Junior Lecturer,CSE

February 26, 2026

1 Parallel Histogram Computation Using Pthreads

1.1 Background: What Is a Histogram?

A **histogram** is a data structure that counts how many times each possible value appears in a dataset. In this lab:

- The histogram is an array where:

$$\texttt{histogram[v]} = \text{number of times value } v \text{ appears in the dataset}$$

1.2 Problem Description

You are given a four-dimensional data array:

$$\texttt{data_array[SUB_ARRAY_COUNT][D][D][D]}$$

Each sub-array is conceptually 3 dimensional and is processed independently by a separate thread.

- Each thread computes a local histogram for one sub-array.
- Local histograms are merged into a **global histogram**.
- Synchronization must be handled correctly to avoid race conditions.

1.3 Constraints

- **Only one mutex is allowed in the entire program.**

1.4 Provided Code: Data Initialization

The following function initializes the data array with pseudo-random values. This code is provided and must not be modified.

```
void init_data_array() {
    srand(0);
    for (int i = 0; i < SUB_ARRAY_COUNT; i++) {
        for (int j = 0; j < SUB_ARRAY_DIM; j++) {
            for (int k = 0; k < SUB_ARRAY_DIM; k++) {
                for (int l = 0; l < SUB_ARRAY_DIM; l++) {
                    data_array[i][j][k][l] = rand() % RANGE;
                }
            }
        }
    }
}
```

1.5 Tasks

1. Create `SUB_ARRAY_COUNT` threads.
2. Assign each thread exactly one sub-array to process.
3. Compute a local histogram inside each thread.
4. Use the **single mutex** to safely merge the local histogram into the global histogram.
5. After all threads complete, verify that the total histogram count equals:

$$\texttt{SUB_ARRAY_COUNT} \times D \times D \times D$$

2 H_2O Bond Formation (Barrier)

2.1 Problem statement

A water molecule is formed by 2 Hydrogen atoms and 1 Oxygen atom. Threads represent individual atoms.

2.2 Rules

- H atoms must wait for another H and one O.
- O atoms must wait for two H atoms.
- They must ”bond” (exit the barrier) simultaneously in groups of three (H-H-O).
- Hint: You can concurrently create H but not O.

2.3 Constraints

- Use **semaphores only**.
- No mutexes.
- Ensure no atom is left behind or grouped incorrectly.

2.3.1 Expected Execution

```
H Ready -> H Ready -> O Ready -> Bond Formed!
```

CSE4502 (Operating Systems Lab): 1A

Jibon Naher[1] and Samnun Azfar[2]

[1]Assistant Professor, CSE
[2]Junior Lecturer, CSE

February 26, 2026

Part A: Process Scheduling Test Case and CPU Time Slice Analysis

Objective: Demonstrate how different CPU scheduling algorithms allocate CPU time slices to processes.

Setup

Clone the following git repository:

```
https://github.com/azfarus/cpu-scheduling-simulation.git
```

Git Link for Scheduling Simulation.

Instructions

- Navigate to the `src/` and `include/` directories.
- You must fill up the boilerplate codes in the **Scheduler.hpp** and **Scheduler.cpp** files to implement the scheduling logic for the required algorithms.
- Pay close attention to **process.hpp** and **process.cpp**. These files contain the `Process` class/struct definition, which includes vital attributes such as Arrival Time (AT), Burst Time (BT), program counter, total instructions and Priority. These values are essential for calculating the execution order and priorities in your implementation.

Test Case (5 Processes)

Process	Arrival Time	Burst Time	Priority
P1	0	5	2
P2	1	4	1
P3	2	2	3
P4	3	6	2
P5	4	3	4

Note: Lower priority number means higher priority.

1. FCFS Scheduling Output

Execution Order and Time Slices:

```
=== Gantt Chart ===
+----------+--------+------+------------+------+
|    P1    |   P2   |  P3  |     P4     |  P5  |
+----------+--------+------+------------+------+
0          5        9      11           17     20
===================
```

2. SJF (Non-Preemptive) Scheduling Output

Execution Order and Time Slices:

```
=== Gantt Chart ===
+----------+------+------+--------+------------+
|    P1    |  P3  |  P5  |   P2   |     P4     |
+----------+------+------+--------+------------+
0          5      7      10       14           20
===================
```

3. Priority Scheduling (Preemptive)

Execution Order and Time Slices:

```
=== Gantt Chart ===
+----------+--------+------------+------+------+
|    P1    |   P2   |     P4     |  P3  |  P5  |
+----------+--------+------------+------+------+
0          5        9            15     17     20
===================
```

4. Pre-emptive SJF Scheduling (Shortest Remaining Time First)

Execution Order and Time Slices:

```
=== Gantt Chart ===
+------+------+------+------+--------+------------+
|  P1  |  P3  |  P1  |  P5  |   P2   |     P4     |
+------+------+------+------+--------+------------+
0      2      4      7      10       14           20
===================
```

5. Round Robin

Execution Order and Time Slices:

```
=== Gantt Chart ===
+------+------+------+------+------+------+------+------+------+------+------+
|  P1  |  P2  |  P3  |  P1  |  P4  |  P5  |  P2  |  P1  |  P4  |  P5  |  P4  |
+------+------+------+------+------+------+------+------+------+------+------+
0      2      4      6      8      10     12     14     15     17     18     20
===================
```

CSE4502 (Operating Systems Lab): 1B

Jibon Naher[1] and Samnun Azfar[2]

[1]Assistant Professor, CSE
[2]Junior Lecturer, CSE

February 26, 2026

Part A: Process Scheduling Test Case and CPU Time Slice Analysis

Objective: Demonstrate how different CPU scheduling algorithms allocate CPU time slices to processes.

Setup

Clone the following git repository:

```
https://github.com/azfarus/cpu-scheduling-simulation.git
```

Git Link for Scheduling Simulation.

Instructions

- Navigate to the `src/` and `include/` directories.
- You must fill up the boilerplate codes in the **Scheduler.hpp** and **Scheduler.cpp** files to implement the scheduling logic for the required algorithms.
- Pay close attention to **process.hpp** and **process.cpp**. These files contain the `Process` class/struct definition, which includes vital attributes such as Arrival Time (AT), Burst Time (BT), program counter, total instructions and Priority. These values are essential for calculating the execution order and priorities in your implementation.

Test Case (5 Processes)

Process	Arrival Time	Burst Time	Priority
P1	0	5	2
P2	1	4	1
P3	2	2	3
P4	3	6	2
P5	4	3	4

Note: Lower priority number means higher priority.

1. FCFS Scheduling Output

Execution Order and Time Slices:

```
=== Gantt Chart ===
+----------+--------+------+------------+------+
|    P1    |   P2   |  P3  |     P4     |  P5  |
+----------+--------+------+------------+------+
0          5        9      11           17     20
===================
```

2. SJF (Non-Preemptive) Scheduling Output

Execution Order and Time Slices:

```
=== Gantt Chart ===
+----------+------+------+--------+------------+
|    P1    |  P3  |  P5  |   P2   |     P4     |
+----------+------+------+--------+------------+
0          5      7      10       14           20
===================
```

3. Priority Scheduling (Non-Preemptive)

Execution Order and Time Slices:

```
=== Gantt Chart ===
+----------+--------+------------+------+------+
|    P1    |   P2   |     P4     |  P3  |  P5  |
+----------+--------+------------+------+------+
0          5        9            15     17     20
===================
```

4. Pre-emptive SJF Scheduling (Shortest Remaining Time First)

Execution Order and Time Slices:

```
=== Gantt Chart ===
+------+------+------+------+--------+------------+
|  P1  |  P3  |  P1  |  P5  |   P2   |     P4     |
+------+------+------+------+--------+------------+
0      2      4      7      10       14           20
===================
```

5. Round Robin

Execution Order and Time Slices:

```
=== Gantt Chart ===
+------+------+------+------+------+------+------+------+------+------+------+
|  P1  |  P2  |  P3  |  P1  |  P4  |  P5  |  P2  |  P1  |  P4  |  P5  |  P4  |
+------+------+------+------+------+------+------+------+------+------+------+
0      2      4      6      8      10     12     14     15     17     18     20
===================
```

CSE4502 (Operating Systems Lab): 2A

Jibon Naher[1] and Samnun Azfar[2]

[1]Assistant Professor, CSE
[2]Junior Lecturer, CSE

February 26, 2026

Part A: Process Scheduling Test Case and CPU Time Slice Analysis

Objective: Demonstrate how different CPU scheduling algorithms allocate CPU time slices to processes.

Setup

Clone the following git repository:

```
https://github.com/azfarus/cpu-scheduling-simulation.git
```

Git Link for Scheduling Simulation.

Instructions

- Navigate to the `src/` and `include/` directories.
- You must fill up the boilerplate codes in the **Scheduler.hpp** and **Scheduler.cpp** files to implement the scheduling logic for the required algorithms.
- Pay close attention to **process.hpp** and **process.cpp**. These files contain the `Process` class/struct definition, which includes vital attributes such as Arrival Time (AT), Burst Time (BT), program counter, total instructions and Priority. These values are essential for calculating the execution order and priorities in your implementation.

Test Case (5 Processes)

Process	Arrival Time	Burst Time	Priority
P1	0	5	2
P2	1	4	1
P3	2	2	3
P4	3	6	2
P5	4	3	4

Note: Lower priority number means higher priority.

1. FCFS Scheduling Output

Execution Order and Time Slices:

```
=== Gantt Chart ===
+----------+--------+------+------------+------+
|    P1    |   P2   |  P3  |     P4     |  P5  |
+----------+--------+------+------------+------+
0          5        9      11           17     20
===================
```

2. SJF (Non-Preemptive) Scheduling Output

Execution Order and Time Slices:

```
=== Gantt Chart ===
+----------+------+------+--------+------------+
|    P1    |  P3  |  P5  |   P2   |     P4     |
+----------+------+------+--------+------------+
0          5      7      10       14           20
===================
```

3. Priority Scheduling (Preemptive)

Execution Order and Time Slices:

```
=== Gantt Chart ===
+----------+--------+------------+------+------+
|    P1    |   P2   |     P4     |  P3  |  P5  |
+----------+--------+------------+------+------+
0          5        9            15     17     20
===================
```

4. Pre-emptive SJF Scheduling (Shortest Remaining Time First)

Execution Order and Time Slices:

```
=== Gantt Chart ===
+------+------+------+------+--------+------------+
|  P1  |  P3  |  P1  |  P5  |   P2   |     P4     |
+------+------+------+------+--------+------------+
0      2      4      7      10       14           20
===================
```

5. Round Robin

Execution Order and Time Slices:

```
=== Gantt Chart ===
+------+------+------+------+------+------+------+------+------+------+------+
|  P1  |  P2  |  P3  |  P1  |  P4  |  P5  |  P2  |  P1  |  P4  |  P5  |  P4  |
+------+------+------+------+------+------+------+------+------+------+------+
0      2      4      6      8      10     12     14     15     17     18     20
===================
```

CSE4502 (Operating Systems Lab): 2B

Jibon Naher[1] and Samnun Azfar[2]

[1]Assistant Professor, CSE
[2]Junior Lecturer, CSE

February 26, 2026

Part A: Process Scheduling Test Case and CPU Time Slice Analysis

Objective: Demonstrate how different CPU scheduling algorithms allocate CPU time slices to processes.

Setup

Clone the following git repository:

```
https://github.com/azfarus/cpu-scheduling-simulation.git
```

Git Link for Scheduling Simulation.

Instructions

- Navigate to the `src/` and `include/` directories.
- You must fill up the boilerplate codes in the **Scheduler.hpp** and **Scheduler.cpp** files to implement the scheduling logic for the required algorithms.
- Pay close attention to **process.hpp** and **process.cpp**. These files contain the `Process` class/struct definition, which includes vital attributes such as Arrival Time (AT), Burst Time (BT), program counter, total instructions and Priority. These values are essential for calculating the execution order and priorities in your implementation.

Test Case (5 Processes)

Process	Arrival Time	Burst Time	Priority
P1	0	5	2
P2	1	4	1
P3	2	2	3
P4	3	6	2
P5	4	3	4

Note: Lower priority number means higher priority.

1. FCFS Scheduling Output

Execution Order and Time Slices:

```
=== Gantt Chart ===
+----------+--------+------+------------+------+
|    P1    |   P2   |  P3  |     P4     |  P5  |
+----------+--------+------+------------+------+
0          5        9      11           17     20
===================
```

2. SJF (Non-Preemptive) Scheduling Output

Execution Order and Time Slices:

```
=== Gantt Chart ===
+----------+------+------+--------+------------+
|    P1    |  P3  |  P5  |   P2   |     P4     |
+----------+------+------+--------+------------+
0          5      7      10       14           20
===================
```

3. Priority Scheduling (Preemptive)

Execution Order and Time Slices:

```
=== Gantt Chart ===
+----------+--------+------------+------+------+
|    P1    |   P2   |     P4     |  P3  |  P5  |
+----------+--------+------------+------+------+
0          5        9            15     17     20
===================
```

4. Pre-emptive SJF Scheduling (Shortest Remaining Time First)

Execution Order and Time Slices:

```
=== Gantt Chart ===
+------+------+------+------+--------+------------+
|  P1  |  P3  |  P1  |  P5  |   P2   |     P4     |
+------+------+------+------+--------+------------+
0      2      4      7      10       14           20
===================
```

5. Round Robin

Execution Order and Time Slices:

```
=== Gantt Chart ===
+------+------+------+------+------+------+------+------+------+------+------+
|  P1  |  P2  |  P3  |  P1  |  P4  |  P5  |  P2  |  P1  |  P4  |  P5  |  P4  |
+------+------+------+------+------+------+------+------+------+------+------+
0      2      4      6      8      10     12     14     15     17     18     20
===================
```

CSE4502 (Operating Systems Lab): 1A

Jibon Naher[1] and Samnun Azfar[2]

[1]Assistant Professor, CSE
[2]Junior Lecturer,CSE

February 26, 2026

Instruction

- Use the PC from the lab
- Total time is 40 minutes
- Submit your phone at the front
- Turn off the internet connection

1 Task: Preepmtive SJF Scheduling Algorithm

1.1 Given Code (Do NOT modify this part)

The following structure and main function are provided. Your implementation must integrate seamlessly with this boilerplate.

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <iomanip>

using namespace std;

struct Process {
    int id;
    int at; // Arrival Time
    int bt; // Burst Time
    int ct; // Completion Time
    int tat; // Turnaround Time
    int wt; // Waiting Time
    int remaining; // Remaining Time for preemption
};

// --- Your Implementation Goes Here ---
void preemptiveSJF(vector<Process>& processes) {
    // Implement the SRTF logic here
}

int main() {
    vector<Process> processes = {
        {1, 0, 6, 0, 0, 0, 6},
        {2, 2, 4, 0, 0, 0, 4},
        {3, 4, 2, 0, 0, 0, 2},
        {4, 6, 5, 0, 0, 0, 5}
    };

```

```
    preemptiveSJF(processes);
    return 0;
}
```

1.2 Implementation Hints

To successfully implement the `preemptiveSJF` function, follow this time-unit simulation approach:

- **Initialize Time:** Start a system clock (e.g., `currentTime = 0`) and track completed processes.
- **At each time unit (increment by 1):**
  1. Find all processes that have arrived (`at <= currentTime`) and are not yet finished (`remaining > 0`).
  2. From these available processes, select the one with the **shortest remaining time (rt)**.
  3. **Tie-breaking:** If multiple processes have the same `rt`, pick the one with the earliest arrival time (`at`).
  4. Execute the selected process for **1 time unit** (decrement its `remaining` time and increment `currentTime`).
  5. **Completion Check:** If a process's `rt` becomes 0:
     - Mark it as completed.
     - Record its CT (current time).
     - Calculate $TAT = CT - AT$ and $WT = TAT - BT$.
  6. **Idle Handling:** If no process has arrived yet, increment the clock until the next arrival.

1.3 Your Task

Complete the `preemptiveSJF` function to fulfill the following requirements:

1. Simulate SRTF using the logic described above.
2. Print a table showing the ID, AT, BT, CT, TAT, and WT for every process.
3. Compute and display the **Average Turnaround Time** and **Average Waiting Time**.

CSE4502 (Operating Systems Lab 9): 1B

Jibon Naher[1] and Samnun Azfar[2]

[1]Assistant Professor, CSE
[2]Junior Lecturer,CSE

February 26, 2026

Instruction: Deadlock Detection in Single-Instance Resource Systems

You will implement an algorithm that analyzes a Resource Allocation Graph (RAG), and check if the system exhibits deadlock.

1 Resource Allocation Graph (RAG)

A RAG is a directed graph used to represent the state of the system:

- Nodes: Two types of nodes exists: Processes ($P = \{P_1, P_2, ...\}$) and Resources ($R = \{R_1, R_2, ...\}$).
- Edges:
  - Request Edge ($P_i \rightarrow R_j$): Process P_i wants resource R_j.
  - Assignment Edge ($R_j \rightarrow P_i$): Resource R_j is currently held by process P_i.

2 Input Specification

Your program will read from standard input (or a file) in the following format:

1. Line 1: Two integers N (number of processes) and M (number of resources).
   - Processes are named 0 to N-1.
   - Resources are named 0 to M-1.
2. Line 2: An integer `E` representing the total number of edges (requests + assignments)
3. Next `E` lines have three values, `u v type`
   - `u`: Source node ID.
   - `v`: Destination Node ID
   - `type`: A string, either "R" (Process u requests Resource v) or "A" (Resource u is assigned to Process v).

2.1 Example Input

```
3 3
5
0 0 R
0 1 A
1 1 R
1 2 A
2 2 R
```

2.2 Interpretation

- P0 requests R0.
- R0 is assigned to P1.
- P1 requests R1.
- R1 is assigned to P2.
- P2 requests R2.

2.3 Output

```
NO DEADLOCK
```

CSE4502 (Operating Systems Lab): 2A

Jibon Naher[1] and Samnun Azfar[2]

[1]Assistant Professor, CSE
[2]Junior Lecturer,CSE

February 26, 2026

Instruction

- Use the PC from the lab
- Total time is 40 minutes
- Submit your phone at the front
- Turn off the internet connection

1 Task: Preepmtive Priority Scheduling Algorithm

1.1 Given Code (Do NOT modify this part)

The following structure and main function are provided. Your implementation must integrate seamlessly with this boilerplate.

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;

struct Process {
    int pid;       // Process ID
    int at;        // Arrival Time
    int bt;        // Burst Time
    int priority;  // Priority (lower number = higher priority)
    int rt;        // Remaining Time
    int ct;        // Completion Time
    int tat;       // Turnaround Time
    int wt;        // Waiting Time
    bool completed; // Completion flag
};

// Function to perform Preemptive Priority Scheduling
void preepmtivePriorityScheduling(vector<Process>& p) {
    // TODO: Implement this function
}

int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;

    vector<Process> processes(n);

    for (int i = 0; i < n; i++) {
```

```
        processes[i].pid = i + 1;
        cout << "Enter Arrival Time, Burst Time, and Priority for Process "
             << processes[i].pid << ": ";
        cin >> processes[i].at >> processes[i].bt >> processes[i].priority;
    }

    // Call the function
    priorityScheduling(processes);

    return 0;
}
```

1.2 Implementation Hints

To successfully implement the `preepmtivePriorityScheduling` function.

- **Initialize:** Set `currentTime = 0` and keep track of completed processes.
- **At each time unit (increment time by 1):**
  1. **Selection:** Among all arrived (`at <= currentTime`) and incomplete (`remaining > 0`) processes, select the one with the **highest priority** (lowest priority value).
  2. **Tie-breaker:** If multiple processes have the same highest priority, pick the process with the **earliest arrival time**.
  3. **Execution:** Execute the selected process for **1 time unit**:
     - Decrement its `remaining` time.
     - Increment the `currentTime`.
  4. **Preemption Logic:** Because we re-evaluate the highest priority process at every single time unit, if a new process arrives with a higher priority (lower number), it will naturally be selected in the next iteration, effectively preempting the current one.
  5. **Completion:** If a process's `remaining` time reaches 0:
     - Record its Completion Time (`ct`).
     - Calculate $TAT = CT - AT$.
     - Calculate $WT = TAT - BT$.
  6. **Idle Handling:** If no process has arrived yet, increment the clock until the next arrival occurs.

1.3 Your Task

Complete the `preepmtivePriorityScheduling` function to fulfill the following requirements:

1. Simulate priority shceduling using the logic described above.
2. Print a table showing the ID, AT, BT, CT, TAT, and WT for every process.
3. Compute and display the **Average Turnaround Time** and **Average Waiting Time**.

CSE4502 (Operating Systems Lab 9): 2B

Jibon Naher[1] and Samnun Azfar[2]

[1]Assistant Professor, CSE
[2]Junior Lecturer,CSE

February 26, 2026

1 System Requirements

You must develop a simulation that accepts user input to define the environment and the execution behavior.

1.1 User Inputs

- Two integers: `N P` where `N` is the number of philosophers and `P` is the phase which can be either `1` or `2`.

2 Phase 1: Deadlock Demonstration

When this phase is selected, the system must demonstrate a state of **Deadlock**.

- This Phase demonstrates deadlock, the code is stuck in a loop and does not end its execution.

3 Phase 2: Resolved Simulation

In this phase, the Deadlock is not present and the code execution terminates.

4 Visualization & Output

- **Real-time State Tracking:** Display whether each philosopher is currently waiting for 2 forks or waiting for 1 or eating.
- **Termination:** Prompt when the system terminates.

CSE4502 (Operating Systems Lab): 1A

Jibon Naher[1] and Samnun Azfar[2]

[1]Lecturer, CSE
[2]Junior Lecturer,CSE

February 26, 2026

1 Part A: File Guard (Conditionals)

1.1 Problem Statement

Develop a Bash script named file_guard.sh that verifies whether a specified file exists and determine whether the provided file name corresponds to a regular file, a directory, or other.

1.2 Logic

1. Take a filename as a command-line argument.
2. Use `if [ -e $filename ]` to check existence.
3. If it exists, check if it is a directory (`-d`) or a file (`-f`).

1.3 Hints

- Use `$1` to access the first argument passed to the script.

1.4 Example Output

Case 1: File exists

```
$ ./file_guard.sh notes.txt
'notes.txt' is a regular file.
```

Case 2: Directory exists

```
$ ./file_guard.sh Documents
'Documents' is a directory.
```

Case 3: File does not exist

```
$ ./file_guard.sh unknown.txt
Error: 'unknown.txt' does not exist.
```

2 Part B : Thread-Safe Histogram

2.1 Problem Statement

A large dataset is processed by multiple threads. They update a shared frequency array (histogram) of size K.

```
histogram[10] = {0, 0, ...}
```

2.2 Rules

- Input is an array of n integers which have values k ranging from $0 \leq k < K$. k is an integer.
- Assume you have $n = 9$. If you create 2 threads then thread $t1$ will calculate the histogram for indices $0 - 4$ and thread $t2$ will calculate the histogram for indices $5 - 8$, both done in a histogram array local to their threads. Then both the threads concurrently update the global histogram using the locally generated histograms.

2.3 Inputs & Outputs

- **Input:** First line of input contains n <space> K.
- **Output:** `Histogram[i]: count` for i in $0..(K-1)$.

2.4 Constraints

- Use **one mutex**.

2.5 Invariant to Preserve

$$\sum \text{histogram}[i] = n$$

CSE4502 Operating Systems Lab 1B

Jibon Naher[1] and Samnun Azfar[2]

[1]Lecturer, CSE
[2]Junior Lecturer,CSE

February 26, 2026

1 Part A: File Size Monitor (Conditionals and Arithmetic)

1.1 Problem Statement

Develop a Bash script named `file_size.sh` that checks the size of a file and classifies it as small, medium, or large based on user-defined thresholds.

1.2 Logic

1. Take a filename as input and verify its existence.
2. Use `size=$(stat -c%s $filename)` to get the file size in bytes.
3. Compare the size against thresholds:
   - $<$ 1000 bytes -> Small file
   - 1000–10000 bytes -> Medium file
   - $>$ 10000 bytes -> Large file
4. Display the classification.

1.3 Example Output

```
$ ./file_size.sh data.log
File: data.log
Size: 15432 bytes
Category: Large file
```

2 Part B: Multi-Level Priority-Based Concurrent Logging

Concurrent threads generate log messages with one of k **priority levels**, denoted as $P_1, P_2, \ldots, P_k$, where P_1 is the **highest priority** and P_k is the **lowest priority**.

The logging system must ensure that all messages of priority P_i are processed and printed **before** any message of priority P_j for all $i < j$, regardless of their arrival order within the critical section.

2.1 Input (Thread Actions)

The system consists of kn concurrent threads:

- n threads producing log messages with priority P_1 (highest priority).
- n threads producing log messages with priority P_2.
- $\vdots$
- n threads producing log messages with priority P_k (lowest priority).

Each thread generates exactly one log message and attempts to insert it into a shared logging buffer.

2.2 Constraints

- $1 \leq n \leq 40 \quad | \quad 2 \leq k \leq 10$.
- You shouldn't sort the output.
- Can you do this without a counter variable or a shared state to track how many threads are done?
- Protect the shared buffer (conceptually equivalent to `stdout` in the case of `printf()`).
- Log insertion and ordering must be thread-safe.
- Messages with higher priority must not be delayed by messages of lower priority that arrive earlier.
- All messages of priority P_i must be printed before any message of priority P_j for all $i < j$.

2.3 Expected Output (Buffer Dump)

After all threads have completed execution, the contents of the logging buffer must satisfy the following ordering:

```
[P1] <Message 1>
[P1] <Message 2>
...
[P1] <Message n1>
[P2] <Message 1>
...
[P2] <Message n2>
...
[Pk] <Message 1>
...
[Pk] <Message nk>
```

The logging buffer is a formal nomenclature. Utilizing basic console prints for the messages is sufficient, as long as each print operation is atomic.

CSE4502 Operating Systems Lab 2A

Jibon Naher[1] and Samnun Azfar[2]

[1]Lecturer, CSE
[2]Junior Lecturer,CSE

February 26, 2026

1 Part A: File Permission Checker (Conditionals and String Operations)

1.1 Problem Statement

Develop a Bash script named `check_permissions.sh` that checks whether a given file is readable, writable, and executable by the current user.

1.2 Logic

1. Take a filename as a command-line argument.
2. Verify if the file exists using `[ -e $filename ]`.
3. If it exists, check each permission using -r, -w or -x
4. Display the permission status clearly.

1.3 Hints

- Use `echo` statements to print messages for each permission type.

1.4 Example Output

```
$ ./check_permissions.sh script.sh
File: script.sh
Readable: Yes
Writable: Yes
Executable: No
```

2 Part B: Concurrent Multi-Account Transfer System

2.1 System Description

Three shared bank accounts are initialized with the following balances:

$$A = 1000, \quad B = 1000, \quad C = 1000$$

Three threads execute concurrently, each performing a fixed number of money transfer transactions between the accounts.

2.2 Workload Specification

The workload consists of the following concurrent threads:

- **Thread T1:** Transfers $Amount_1$ units from account A to account B, repeated $Repeat_1$ times.
- **Thread T2:** Transfers $Amount_2$ units from account B to account C, repeated $Repeat_2$ times.
- **Thread T3:** Transfers $Amount_3$ units from account C to account A, repeated $Repeat_3$ times.

Each repetition is considered a distinct transaction. Each $Amount_i$ and $Repeat_i$ are integers. And the relationship between $Amount_i$ and $Repeat_i$ is given as $Amount_i * Repeat_i = 1000$ and $Amount_i \mid 1000$ that is 1000 is divisible by $Amount_i$.

2.3 Correctness Constraints

The system must satisfy the following constraints:

- Each transaction must be **atomic** (all-or-nothing).
- No account balance may ever become negative.
- Concurrent execution must not violate consistency.

2.4 Expected Input, Output and Execution Behavior

2.5 Input

Input should be of 3 integers, as such:

```
Amount1 Amount2 Amount3
```

And each of the input integers satisfy the constraint $Amount_i * Repeat_i = 1000$.

2.5.1 Transaction Logging

For every transaction, the system must generate logs indicating the completion of the transfer.

```
[Tk] Transferred <amount> from <source> to <destination>
```

2.5.2 Intermediate Account States

After each thread completes all of its assigned transactions, the system must print the current account balances:

```
After T1 completes: A = x1, B = y1, C = z1
After T2 completes: A = x2, B = y2, C = z2
After T3 completes: A = x3, B = y3, C = z3
```

After each thread completes the expression $A + B + C = 3000$ must hold

2.5.3 Final System State

After all threads have completed execution, the final account balances must be:

```
Final Balances:
A = 1000
B = 1000
C = 1000
```

CSE4502 Operating Systems Lab 2B

Jibon Naher[1] and Samnun Azfar[2]

[1]Asst. Professor, CSE
[2]Junior Lecturer,CSE

February 26, 2026

1 Part A: File Type Identifier (Case and Conditionals)

1.1 Problem Statement

Create a Bash script named `file_type.sh` that determines the type of a file based on its extension (e.g., .txt, .sh, .c, .jpg).

1.2 Logic

1. Accept a filename as a command-line argument.
2. Use a `case` statement to identify the file type.
3. Print the result based on the extension.

1.3 Hints

- Extract the file extension using parameter expansion:
  `ext="$filename##*."`.
- Convert extension to lowercase for case-insensitive matching using :
  `ext_lower=$(echo "$ext"` tr '[:upper:]' '[:lower:]') |

1.4 Example Output

```
$ ./file_type.sh photo.jpg
File type: Image File

$ ./file_type.sh report.txt
File type: Text File

$ ./file_type.sh main.c
File type: C Source File
```

2 Part B: Sequential Character Printing Using Semaphores

2.1 Problem Statement

Given an integer n such that $1 \leq n \leq 26$, create n concurrent threads. Each thread is responsible for printing exactly one uppercase character from the English alphabet. Despite starting in an arbitrary order, the threads must print characters sequentially in alphabetical order.

2.2 Thread Responsibilities

- Thread 1 prints character `A`
- Thread 2 prints character `B`
- $\vdots$
- Thread n prints character corresponding to `('A' + n - 1)`

Each thread prints exactly one character and then terminates.

2.3 Synchronization Constraints

- Threads may start execution in any order.
- Output must always appear in strictly increasing alphabetical order.
- Each thread waits a random amount of time before printing, The execution of the threads should be sequential even after applying random delays. Use the following snippet to introduce delays:

```
void delay_ms() {
    unsigned int milliseconds = (rand() * 100)%500;
    usleep(milliseconds * 1000);  // Convert ms to microseconds
}
```

2.4 Expected Output

For a given value of n, the output must be:

```
A B C ...
```

ending at the n-th character.

2.5 Example

For $n = 4$, four threads are created and may start in any order, but the output must be:

```
A B C D
```